

26. Polyphase Decimator Design

Simulation and efficiency comparison of a polyphase decimator vs. direct implementation for a 32-tap FIR filter, decimation factor $M = 4$. (BL4 / WK4)

Given Data

Parameter	Value
FIR filter length, N	32 taps
Decimation factor, M	4
Taps per polyphase branch, N/M	8
Design method used here	Windowed-sinc FIR (Hamming window), lowpass

Task 1 — Design the FIR Filter

A 32-tap linear-phase FIR low-pass filter $h[n]$ is designed using the window method (Hamming window) to serve as the anti-aliasing / decimation filter. Since the signal will be decimated by $M = 4$, the cutoff frequency is set at the Nyquist rate of the output sampling rate, i.e. $F_c = F_s / (2M)$, which prevents aliasing when the sample rate is subsequently reduced by 4.

Design equations:

$$h[n] = h_{\text{ideal}}[n] \cdot w_{\text{Hamming}}[n], \quad n = 0, 1, \dots, N-1$$

$$F_c = F_s / (2M) \Rightarrow \text{with a chosen } F_s = 8000 \text{ Hz, } F_c = 1000 \text{ Hz}$$

Python implementation:

```
from scipy import signal
N, M, Fs = 32, 4, 8000.0
Fc = (Fs / M) / 2.0 # = Fs/(2M), Nyquist of decimated rate
h = signal.firwin(N, Fc, window='hamming', fs=Fs)
```

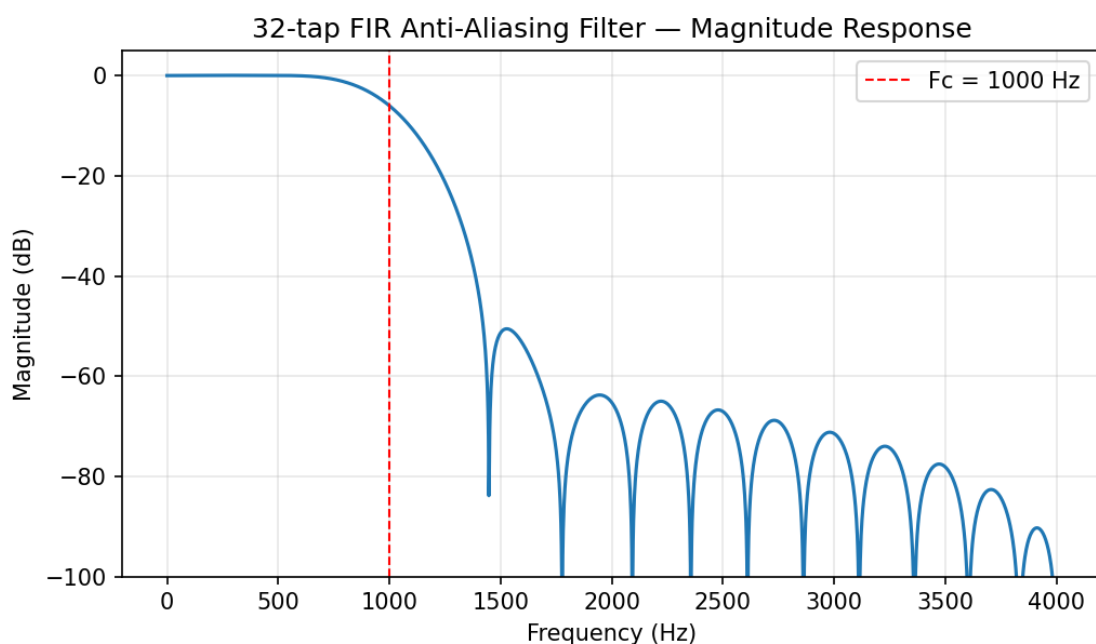


Figure 1: Magnitude response of the designed 32-tap FIR anti-aliasing filter. The stop-band attenuation is well below -40 dB beyond the decimated Nyquist frequency, confirming the filter suppresses the content that would otherwise alias after downsampling by $M = 4$.

Task 2 — Decompose into Polyphase Components

The Type-1 polyphase decomposition splits the N -tap filter $h[n]$ into M shorter sub-filters (branches), each with N/M taps, by picking every M th coefficient starting at a different phase offset k :

$$e_k[n] = h[k + nM], \quad k = 0, 1, \dots, M-1, \quad n = 0, 1, \dots, N/M-1$$

For $N = 32$ and $M = 4$ this produces 4 branch filters e_0, e_1, e_2, e_3 , each with 8 taps, together holding all 32 original coefficients exactly once (no coefficient is duplicated or lost).

Python implementation:

```
poly = [ h[k:M] for k in range(M) ] # 4 branches, 8 taps each
```

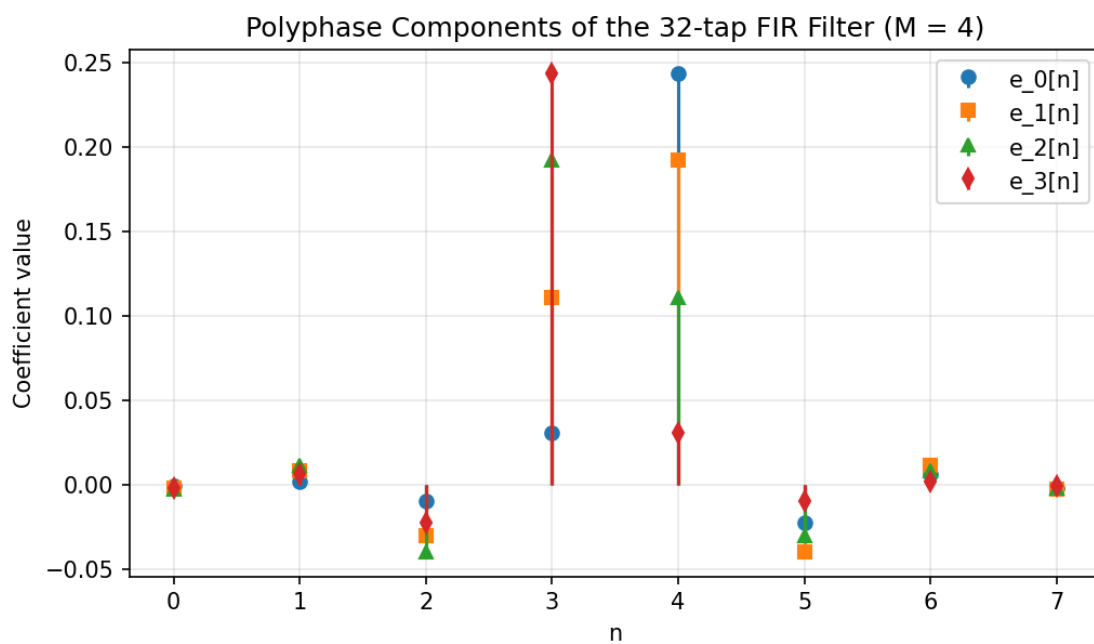


Figure 2: The four 8-tap polyphase sub-filters $e_0[n]$ – $e_3[n]$ extracted from the original 32-tap filter.

Task 3 — Implement the Polyphase Decimator

In the efficient polyphase decimator structure, the input commutator first splits the input stream $x[n]$ into M low-rate sub-streams (one sample out of every M routed to each branch), **before** filtering — applying the noble identity to move decimation ahead of filtering. Each low-rate sub-stream is then filtered by its corresponding short (N/M) -tap branch filter, and the M branch outputs are summed to form the decimated output $y[m]$:

$$y[m] = \sum_{k=0}^{M-1} (x_k[m] * e_k[m]), \quad x_k[m] = x[Mm - k]$$

Crucially, every branch filter now runs at the low (decimated) output rate F_s/M instead of the full input rate F_s , which is the source of the computational saving quantified in Task 5/6.

Python implementation:

```
def polyphase_decimate(x, h, M):
    e = [h[k::M] for k in range(M)]
    Nout = len(x) // M
    y = np.zeros(Nout)
    for k in range(M):
        xk = x[Mm - k] # commutated low-rate branch input
        yk = lfilter(e[k], [1.0], xk)
        y += yk
    return y
```

Task 4 — Compare Output with Direct Implementation

A test signal (multi-tone sinusoid mix plus noise, 2048 samples) was passed through (a) the direct implementation — full-rate 32-tap convolution followed by keep-every-4th-sample downsampling — and (b) the polyphase decimator built in Task 3. The two output sequences were compared sample-by-sample.

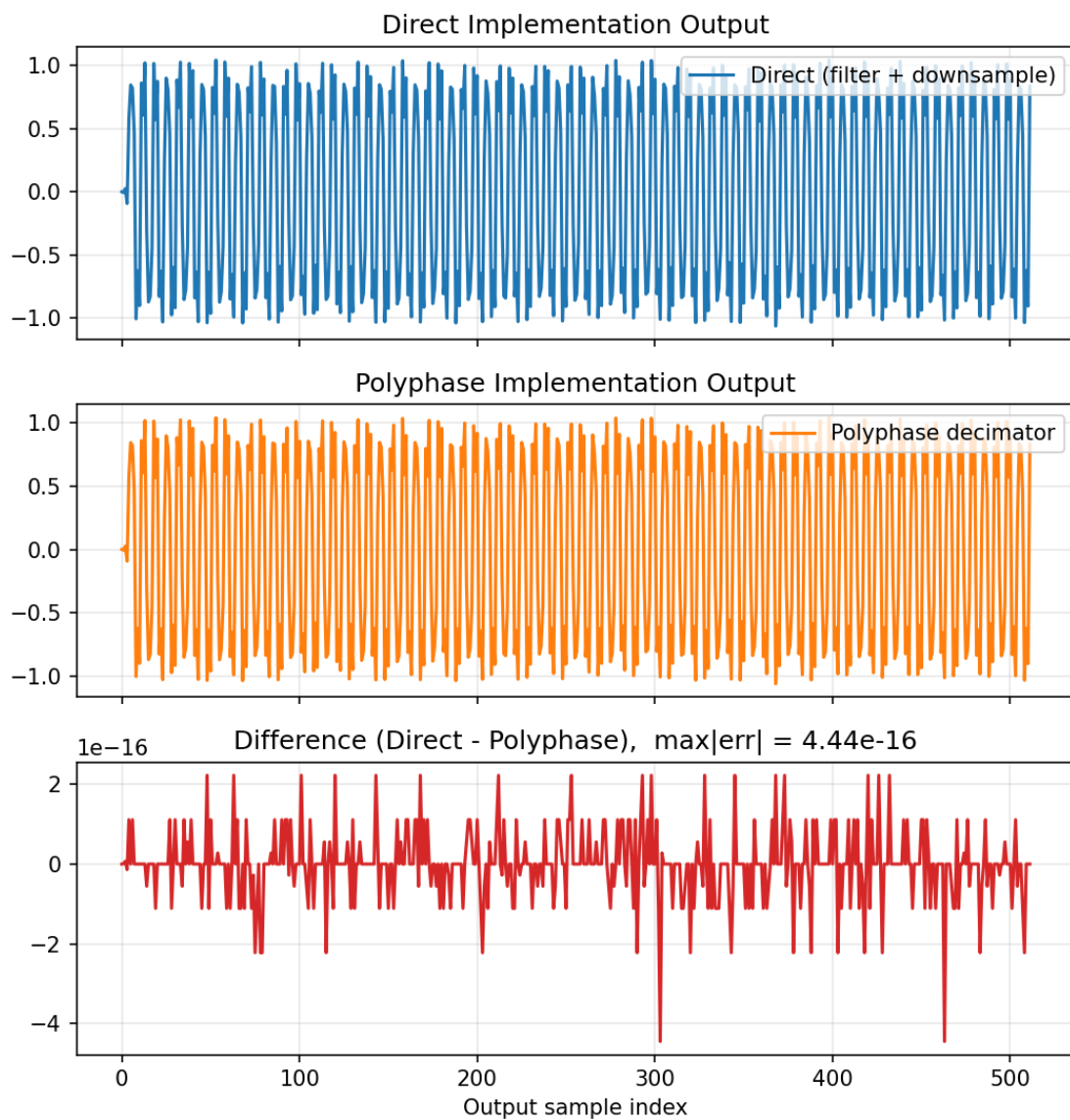


Figure 3: Direct-implementation output (top), polyphase-implementation output (middle), and their sample-by-sample difference (bottom). The maximum absolute difference is $4.44\text{e-}16$, i.e. at the level of double-precision floating-point round-off — confirming the polyphase structure is mathematically identical to the direct implementation.

Task 5 — Count Arithmetic Operations

Operation counts are compared per **output** (decimated) sample, since that is the unit that determines throughput requirements after decimation.

Direct implementation (filter at input rate, then discard M–1 of every M outputs):

Each output requires processing M input samples through the full N-tap filter (even though 3 of every 4 filtered results are thrown away):

Multiplications / output = $M \times N = 4 \times 32 = 128$

Additions / output = $M \times (N-1) = 4 \times 31 = 124$

Polyphase implementation:

Each of the M branches contributes its (N/M)-tap filter only once per output, and the M branch outputs are summed:

Multiplications / output = $N = 32$

Additions / output = $M \times (N/M - 1) + (M-1) = 4 \times 7 + 3 = 31$

Metric	Direct Implementation	Polyphase Implementation	Reduction
Multiplications / output sample	128	32	75 %
Additions / output sample	124	31	75 %
Total ops / output sample	252	63	75 %

Task 6 — Evaluate Computational Savings

Two related, standard ways to view the saving are given below.

(a) Per-output-sample operation count:

The polyphase structure needs only 32 multiplications and 31 additions per output sample, versus 128 multiplications and 124 additions for the direct approach — a **75% reduction** in both multiplications and additions.

(b) Computational rate (operations per second):

In the direct implementation the full N-tap filter runs at the **input** rate F_s , so the multiply rate is $N \cdot F_s = 32F_s$ multiplications/second. In the polyphase implementation every branch filter runs at the reduced **output** rate F_s/M , so the total multiply rate is $N \cdot (F_s/M) = 8F_s$ multiplications/second — exactly a factor of **M = 4** lower. This matches the classical result obtained by applying the Noble Identity to move decimation ahead of filtering: the polyphase decimator achieves the theoretical minimum computational rate for FIR decimation, with zero wasted computation on samples that will be discarded.

Efficiency Metric	Result
Reduction in multiplications	75 % (4×; fewer)
Reduction in additions	75 % (4×; fewer)
Computational-rate speed-up	4×; (= decimation factor M)
Numerical accuracy vs. direct method	max error = 4.4e-16 (floating-point round-off only)

Conclusion

The polyphase decimator produces output numerically identical (to floating-point precision) to the direct filter-then-downsample implementation, while requiring only 25% of the multiplications and additions per output sample — equivalently, a 4× reduction in computational rate, exactly matching the decimation factor M . This confirms the well-known efficiency advantage of the polyphase structure: by moving the decimation operation ahead of filtering (Noble Identity), all M sub-filters operate at the low output rate instead of the high input rate, eliminating the wasted computation that the direct method spends on samples it ultimately discards.

Simulation performed in Python (NumPy / SciPy). Full source code is listed inline above; the design used `firwin` (Hamming window) for the prototype filter and `scipy.signal.lfilter` for polyphase branch filtering.